

MANTIS: Metacognitive Adaptive Network with Tiered Inference Strategies for Hallucination Mitigation

Anonymous Authors
Institution Name
email@institution.edu

Abstract

Large Language Models (LLMs) exhibit a fundamental trade-off between long-context retention, computational efficiency, and hallucination rates. Existing approaches either scale quadratically with context length (dense transformers), compress information lossy (state-space models), or lack contextual statefulness (retrieval-augmented generation). We propose **MANTIS** (Metacognitive Adaptive Network with Tiered Inference Strategies), a hybrid architecture that couples a sparse Mixture-of-Experts (MoE) backbone with a meta-controlled three-tier memory system. We present a complete reference implementation in which a *Meta-Controller*, trained with reinforcement learning (PPO), decides for each query whether to bypass the optional components, whether to read (2) episodic memory (Mamba-based SSM retrieval keys over recent interactions) and (3) semantic memory (a FAISS vector index), whether to add a bounded bias to expert selection, and whether to verify the answer. A critic scores the draft against the retrieved evidence, retrieves once more when the draft fails, and abstains when the evidence still does not support an answer. This paper describes the architecture, its five-stage training pipeline and its evaluation harness as a framework for research into adaptive memory systems. No model has been trained yet, so its expected benefits are hypotheses; we state the measurements that would test them.

1 Introduction

The success of Large Language Models [1, 2, 5] has been accompanied by persistent challenges in long-context reasoning and factual accuracy. Current architectures face three fundamental limitations:

(1) Quadratic Scaling: Self-attention in transformers costs $O(L^2)$ in sequence length L [6, 7]. Long windows are a matter of training and serving budget rather than a hard limit; our presets fix an 8K window as a design choice.

(2) Lost-in-the-Middle: Dense attention struggles to maintain uniform retrieval accuracy across extended contexts, exhibiting U-shaped performance curves where mid-sequence information is preferentially forgotten [8].

(3) Hallucination Persistence: Despite advances in pre-training scale and alignment, LLMs continue to generate plausible but factually incorrect outputs, particularly when forced to extrapolate beyond training distributions [9, 10].

Existing mitigation strategies exhibit complementary weaknesses. *Retrieval-Augmented Generation* (RAG) [11] offloads long-term memory to external datastores but lacks stateful compression of conversation context. *State Space Models* (SSMs) [12, 13] achieve linear complexity but sacrifice the ability to perform targeted retrieval over historical context. *Sparse attention* patterns [14, 15] reduce compute (Longformer’s sliding-window pattern scales linearly in L) but give up full pairwise attention over the history.

1.1 Contributions

We propose MANTIS, a metacognitive architecture that integrates three memory tiers under unified RL-based control:

1. **Sparse MoE Backbone:** A base configuration with 6.7B parameters, 8 experts, top-2 routing and grouped-query attention, activating about 1.9B parameters per token.

2. **Three-Tier Memory Hierarchy:**

- *L1 (Working Memory):* Causal self-attention with rotary position embeddings over the preset’s window (8K tokens for base).

- *L2 (Episodic Memory)*: Stacked Mamba SSM blocks that encode each stored interaction into a 256-dimensional retrieval key; entries keep their tokens, provenance and hit counts.
 - *L3 (Semantic Memory)*: A FAISS vector index with namespaces, trust levels and approximate retrieval over up to 1M entries.
3. **Meta-Controller Architecture**: A 105M-parameter residual MLP policy, trained with PPO, that routes queries based on the query embedding, query predictability and memory state.
 4. **Integrated Critic Model**: An 80M-parameter encoder over the frozen backbone’s hidden states that scores responses against retrieved evidence, triggers one bounded round of evidence recovery and, failing that, abstention.

We describe the system architecture and its implementation, state its costs, and list the measurements that would establish a benefit over the same backbone with ordinary retrieval.

2 Related Work

Sparse Models & Mixture of Experts. Switch Transformer [16] and Mixtral [17] demonstrate that sparse expert activation maintains model quality while reducing per-token compute. Our MoE backbone follows top-K routing principles but integrates with memory-augmented components rather than serving as the sole architecture.

State Space Models. Structured SSMs (S4 [12], Mamba [13], Jamba [18]) achieve linear complexity through selective state compression. While Jamba explores SSM-attention hybrids at the layer level, MANTIS implements metacognitive routing with hierarchical memory and explicit control over inference strategies.

Memory-Augmented Networks. Neural Turing Machines [19] and Memory Networks [20] pioneered external memory interfaces. Modern RAG systems [11, 21] combine dense retrievers with generative models. MANTIS extends this paradigm with tiered inference strategies and learned metacognitive access policies.

Hallucination Detection. Recent work addresses factual errors through self-consistency checking [22], retrieval verification [23], and uncertainty quantification [24]. Our critic model explicitly conditions on retrieved evidence for calibrated confidence estimation.

3 Architecture

3.1 Notation and Problem Setup

Given an input sequence $\mathbf{x} = (x_1, \dots, x_L)$ where $x_i \in \mathcal{V}$ (vocabulary), we seek to model the conditional distribution $p(x_{t+1} | x_{\leq t})$ while maintaining three objectives:

$$\text{Minimize: } \mathcal{L}_{LM} = - \sum_{t=1}^L \log p(x_t | x_{<t}) \quad (\text{Language Modeling}) \quad (1)$$

$$\text{Minimize: } \mathcal{C}_{compute}(x) \quad (\text{Computational Cost}) \quad (2)$$

$$\text{Maximize: } \mathcal{Q}_{factual}(x) \quad (\text{Factual Accuracy}) \quad (3)$$

Let d_{model} denote embedding dimension, $N = 8$ the number of experts, and $K = 2$ the activation sparsity.

3.2 Backbone

The base model is a decoder-only transformer with pre-norm LayerNorm blocks. Attention uses rotary position embeddings [29] and fused scaled dot-product attention over a window of L_{max} tokens, with a key/value cache for generation. Grouped-query attention [4] shares each key/value head among several query heads: the `base` preset uses 32 query heads and 8 key/value heads, so one 8K-token cache holds about 0.375 GiB in 16-bit precision instead of 1.5 GiB. Input embeddings and the output projection share weights. Every block’s feed-forward sublayer is the MoE layer below; with $N = 1$ it reduces to a dense feed-forward network. Table 1 lists the implemented configurations.

Tokenizer. The implementation uses a 512-token vocabulary matched longest-first through a trie. It holds 283 domain tokens for the text protocol of a synthetic ecological simulator, which the project uses as its first training corpus, plus 18 extra ASCII characters, 161 UTF-8 byte-fallback tokens and 50 reserved slots. Numbers are split into digits. Any string round-trips losslessly, but natural-language text falls back to single characters and bytes, so its sequences are several times longer than under a subword vocabulary.

Table 1: Model presets in the reference implementation (vocabulary of 512 tokens, $L_{max} = 8192$). Heads are query/key-value heads. The controller and critic scale with the preset.

Preset	d_{model}	Layers	Heads	Experts	Parameters	Active	Controller	Critic
micro	256	4	4/4	1 (dense)	3.3M	3.3M	0.6M	2.2M
tiny	512	6	8/4	4	55M	30M	2.4M	14M
small	1024	12	32/8	4	435M	234M	18M	45M
base	2048	24	32/8	8	6.70B	1.86B	106M	80M

3.3 Base Mixture-of-Experts Layer

Each MoE layer consists of N expert networks $\{E_i\}_{i=1}^N$ where $E_i : \mathbb{R}^{d_{model}} \rightarrow \mathbb{R}^{d_{model}}$ is a two-layer feed-forward network:

$$E_i(\mathbf{x}) = W_{2,i} \text{GELU}(W_{1,i}\mathbf{x} + \mathbf{b}_{1,i}) + \mathbf{b}_{2,i} \quad (4)$$

where GELU is the Gaussian Error Linear Unit activation [25].

The gating network $G(\mathbf{x})$ computes routing probabilities:

$$\mathbf{h} = W_g \mathbf{x} + \mathbf{b}_g + \boldsymbol{\beta} \in \mathbb{R}^N \quad (5)$$

$$\mathbf{p} = \text{Softmax}(\mathbf{h}) \quad (6)$$

where $\beta_\ell = s \tanh(\mathbf{r}_\ell)$ is the expert bias the Meta-Controller chooses for layer ℓ and the whole query, with $s = 2$ and raw output $\mathbf{r}_\ell$. The bias is zero at initialization and off by default; how much of the learned routing survives depends on its magnitude, which the bound limits to $\pm s$ per logit. Expert intervention stays disabled until an ablation shows that it improves held-out quality at a measured cost.

Top- K routing selects the expert subset $\mathcal{T} = \text{TopK}(\mathbf{p}, K = 2)$ and renormalizes:

$$\mathbf{y} = \sum_{i \in \mathcal{T}} \frac{p_i}{\sum_{j \in \mathcal{T}} p_j} E_i(\mathbf{x}) \quad (7)$$

Load Balancing. To prevent expert collapse, each MoE layer adds the Switch Transformer auxiliary loss [16]:

$$\mathcal{L}_{bal} = \alpha \cdot N \sum_{i=1}^N f_i \cdot \bar{p}_i \quad (8)$$

where f_i is the fraction of tokens in the batch whose top-1 expert is i , $\bar{p}_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} p_i(x)$ is the average routing probability across the batch, and $\alpha = 0.01$. The model sums $\mathcal{L}_{bal}$ over its layers. Because f_i counts only first choices, each layer also reports the fraction of top- K assignments per expert, the load actually dispatched. Experts run in a per-expert loop; sparse parameter counts do not by themselves give fast execution, so kernel work (grouped GEMMs, expert parallelism) should follow profiling.

3.4 Meta-Controller

The Meta-Controller π_θ sees one vector per query, so it is a residual MLP rather than a sequence model: 6 pre-norm feed-forward blocks (inner width 4096; 105M parameters at $d_{model} = 2048$; smaller presets use 2–4 blocks). The base model first runs a forward pass over the query, and that pass is kept: its key/value cache and hidden states serve decoding and the memory write later, so the query is encoded once. Its mean-pooled final hidden state is the query embedding $\mathbf{e}_q$, and its next-token distributions yield the features of the state summary:

$$\mathbf{s}_t = \text{Enc}_\psi(u_t, \ell_t, \mathbf{m}_t, c_t) \quad (9)$$

where:

- u_t : mean next-token entropy over the query, normalized by $\log |\mathcal{V}|$ (how predictable the query text is, not whether the answer will be correct)
- ℓ_t : query length relative to the attention window

- $\mathbf{m}_t$: episodic fill (entries over capacity) and whether semantic memory holds any entry
- c_t : mean top-1 next-token probability (confidence)

Enc_ψ is a small learned encoder that projects these features to a 128-dimensional summary.

The controller computes $\mathbf{z}_t = \text{MLP}_\theta([\mathbf{e}_q; \mathbf{s}_t])$ and outputs four gate probabilities and an expert bias:

$$g_{byp} = \sigma(W_{byp}\mathbf{z}_t) \quad (\text{Bypass}) \quad (10)$$

$$g_{epi} = \sigma(W_{epi}\mathbf{z}_t) \quad (\text{Episodic Access}) \quad (11)$$

$$g_{sem} = \sigma(W_{sem}\mathbf{z}_t) \quad (\text{Semantic Access}) \quad (12)$$

$$g_{ver} = \sigma(W_{ver}\mathbf{z}_t) \quad (\text{Verification}) \quad (13)$$

$$\boldsymbol{\mu}_{exp} = W_{exp}\mathbf{z}_t \quad (\text{Expert Bias, raw, one row per layer}) \quad (14)$$

where σ is the sigmoid function.

Stochastic policy. During RL, each gate is a Bernoulli variable with probability g_* , and the raw expert bias is drawn from $\mathcal{N}(\boldsymbol{\mu}_{exp}, \text{diag}(\boldsymbol{\sigma}^2))$ with a learned $\boldsymbol{\sigma}$ (initialized to e^{-1}); the applied bias is $s \tanh(\cdot)$ of the sample. At inference the policy is deterministic: $a_* = \mathbb{I}[g_* > \tau]$ with $\tau = 0.5$, and the bias uses $\boldsymbol{\mu}_{exp}$. An action mask keeps a gate out of the policy log-probability when its component is missing (no episodic memory, semantic store or critic) or when the bypass path ignored it, so unused actions add no credit-assignment variance.

Routing. The bypass path requires both an open bypass gate and a predictable query ($u_t < 0.2$). It decodes from the query’s cached prefill with no memory reads, no expert bias and no verification; the backbone still runs at full depth, so bypass saves the optional components, not backbone work. Otherwise the open memory gates retrieve candidate evidence, a hybrid reranker (tier similarity plus word-level F1 against the query) selects items under a token budget with an equal share per tier and leftover flow between tiers, and the selected lines, each tagged with a source identifier and provenance label, are prepended to the query. Decoding applies β when expert intervention is enabled; the cached prefill is reused whenever the prompt is unchanged. Input beyond the window is ingested into memory as document chunks before the query is answered. A fixed route policy (all gates open, all closed, or bypass) replaces the learned one for ablations.

3.5 Episodic Memory: Selective State Space Model

The L2 episodic memory implements a selective SSM following the Mamba architecture [13]. For input sequence $\{\mathbf{x}_t\}$, the continuous-time state-space equations are:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t) \quad (15)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{h}(t) \quad (16)$$

Selective Discretization. We compute input-dependent time-scales:

$$\Delta_t = \text{Softplus}(W_\Delta \mathbf{x}_t + \mathbf{b}_\Delta) \quad (17)$$

The continuous parameters are discretized via zero-order hold:

$$\bar{\mathbf{A}}_t = \exp(\Delta_t \mathbf{A}) \quad (18)$$

$$\bar{\mathbf{B}}_t = (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B} \quad (19)$$

The discrete recurrence becomes:

$$\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t \quad (20)$$

$$\mathbf{y}_t = \mathbf{C}_t \mathbf{h}_t + \mathbf{D}\mathbf{x}_t \quad (21)$$

where $\mathbf{B}_t = W_B \mathbf{x}_t$ and $\mathbf{C}_t = W_C \mathbf{x}_t$ provide input-dependent selectivity.

Implementation. The module stacks 8 Mamba blocks (state size 256, expansion 2, convolution width 4) over the base model’s hidden states. Mean pooling and a linear projection reduce each interaction of up to 8K tokens to a 256-dimensional key. Each entry is encoded independently: the module is learned retrieval over interactions, not a recurrent state carried across turns, and a single key can miss names, numbers, negations and updates spread over entries.

An entry holds its token segments (query and response, or a document chunk), its key, a pooled d_{model} embedding for reranking, a hit count and provenance (namespace, source, trust, timestamp); the full hidden states are not retained, so E entries cost $O(E(d_{state} + d_{model}))$ plus their tokens. The engine writes every answered interaction, reusing the hidden states computed during decoding, and never writes an abstention. The buffer keeps the latest 100 entries; an entry evicted at capacity is handed to the consolidator before it leaves. Retrieval ranks entries of the caller’s namespace by cosine similarity of keys and returns the top 3 with their text.

3.6 Semantic Memory: Vector Index Retrieval

The L3 semantic memory uses a FAISS index [26] in which every vector keeps a stable ID:

- **Embedding:** Each entry is the mean-pooled base-model embedding of its text, passed through a linear projection P (trained in Stage 2) and L2-normalized, so exact L2 ranking equals cosine ranking. The store records a fingerprint of the tokenizer and full backbone weights that produced its vectors, and the engine refuses a store built by a different model.
- **Provenance:** Entries carry a namespace (per caller, plus a shared `global` space), a source and a trust level (user-provided or external text 2, verified model output 1, unverified model output 0), a timestamp and supersession links. Retrieval filters by namespace and trust; with the default minimum trust of 1, an unverified generated claim is never cited as a fact, and a superseded entry stays as a source record but is no longer returned.
- **Indexing:** Below 10K entries the index searches exhaustively. From 10K entries it trains an inverted file (IVF) over $n_{list} = \min(4\sqrt{M}, M/39)$ Voronoi cells and probes $\min(n_{list}, 16)$ of them per query. Retrieval is then approximate: recall depends on the embedding distribution, the partitioning, the quantization and the probe count, and `recall_at_k()` measures it against exact search.
- **Quantization:** Product quantization with 64 sub-quantizers of 8 bits compresses each 2048-dimensional vector into 64 bytes inside the index. The store keeps the full FP32 vectors alongside for rebuilds, so 1M entries need 8.2 GB of RAM before text, metadata and index structures.
- **Eviction:** Beyond capacity the oldest entries are evicted. Evicted and deleted IDs become tombstones; searches over-fetch by a bounded amount to skip them, and the index is rebuilt off-thread, then swapped in, once more than 20% of it is stale. A rebuild retrains the IVF-PQ codebooks over all live vectors.

For query $\mathbf{q}$, retrieval proceeds as:

$$\mathcal{R}_k(\mathbf{q}) = \text{TopK}_{\mathbf{v} \in \mathcal{I}} \left(\frac{(P\mathbf{q})^\top (P\mathbf{v})}{\|P\mathbf{q}\| \|P\mathbf{v}\|} \right) \quad (22)$$

with $k = 5$ at inference. Scanning n_{list} centroids and 16 cells of about M/n_{list} vectors each costs $O(\sqrt{M})$ under fixed dimensionality and probe count and roughly balanced cells; tombstone over-fetch and rebuilds add to that. Datastore capacity is not context: what the generator sees is the evidence that retrieval finds and the token budget admits.

Consolidation. When the engine holds both memories it starts a consolidator with a bounded work queue. Every entry evicted from L2 is written to L3 as its own record, with its text, its pooled embedding (no new backbone pass), its provenance and a link to its episodic origin, so nothing leaves the buffer without a durable copy; a full queue falls back to a synchronous write. A periodic cycle (every 10 minutes by default) promotes entries that retrieval has already found useful (hit count ≥ 1) or that carry an `important` flag, and removes from L2 only those whose write succeeded. Exact duplicates of recently stored text are not written twice. Closing the engine flushes the queue and checkpoints both stores.

3.7 Critic Model for Hallucination Detection

The critic C_ϕ is a 6-layer pre-norm transformer encoder ($d_{model} = 1024$, 16 heads; 80M parameters at the `base` preset) over the frozen backbone’s final hidden states of the retrieved evidence $\mathcal{E}$, the query $\mathbf{q}$ and the generated response $\mathbf{r}$:

$$P_{ver} = \sigma(C_\phi(h_{base}([\mathcal{E}; \mathbf{q}; \mathbf{r}])) / T) \quad (23)$$

Reading backbone representations gives the critic the backbone’s language knowledge instead of asking a randomly initialized encoder to learn language from a small labeled set. Evidence comes first so that, under the causal backbone,

every response token has seen the evidence and the query. Learned segment embeddings mark the three parts, and the input fits a 2048-token budget: the response gets up to half, the evidence up to 35% and the query the last 15%, with unused capacity flowing to the response, then the evidence, then the query. Mean pooling feeds one sigmoid head; T is a temperature fitted on a held-out calibration split in Stage 4. The critic receives the same evidence lines the generator saw, episodic and semantic alike. When $P_{ver} < 0.6$, the engine retrieves once more, now conditioning on the query and the rejected draft, regenerates and rescores; if the score still falls short (or no memory is available) it replaces the response with an abstention, which is never written to memory.

Training minimizes binary cross-entropy on the correctness logit. Calibration is measured, not assumed: Stage 4 reports Brier score and expected calibration error on a validation split that the temperature fit did not see.

4 Proposed Training Methodology

Training runs in five stages. Stage 1 is required; Stages 2, 4 and 5 each need only the Stage 1 model, and Stage 3 enables whichever gates the Stage 2 and Stage 4 outputs support. Stages 2–4 leave the generator untouched; Stage 5 is the one that teaches it the runtime prompt format.

4.1 Stage 1: Pre-training

The base MoE model is trained alone on language modeling:

$$\mathcal{L}_{total} = \mathcal{L}_{LM} + \sum_{layers} \mathcal{L}_{bal} \quad (24)$$

The implementation uses AdamW [27] ($\beta = (0.9, 0.95)$, weight decay 0.01) with a peak learning rate of 3×10^{-4} , linear warmup, cosine decay to 10% of the peak, and FP16 or BF16 mixed precision. 8-bit AdamW, gradient checkpointing and DeepSpeed ZeRO-2 are optional. Documents end with an EOS token and are packed into windows of $L + 1$ tokens. For natural-language pre-training we propose a corpus of about 2T tokens from web text, books and code.

4.2 Stage 2: Memory Fine-Tuning

With the base model frozen, Stage 2 trains the episodic SSM and the semantic projection P on (query, context) pairs. Both use InfoNCE over in-batch negatives with temperature 0.07: the SSM state of a query must match the state of its context, and the projected embedding of a query must retrieve its context. The losses are weighted 1.0 (episodic) and 0.5 (semantic), with learning rate 10^{-5} . Afterwards every context is written to an L3 store, in the shared namespace with full trust and stamped with the base model’s embedding fingerprint, for Stage 3 and inference.

4.3 Stage 3: Meta-Controller Reinforcement Learning

The Meta-Controller and the state encoder are optimized with Proximal Policy Optimization (PPO) [28]. Each query is one episode and the objective is the expected reward of a single routing decision:

$$J(\theta) = \mathbb{E}_{q, a \sim \pi_{\theta}(\cdot|q)} [R(q, a)] \quad (25)$$

Episodes are independent only while memory is fixed. Training episodes write their interactions to memory, so training is a sequential, stateful process, and the reward assigns no credit for the future usefulness of what an episode stored. Validation and the supervised warm start below run on frozen memory (no writes, no consolidation).

Reward Function:

$$R = w_1 r_{acc} - w_2 r_{lat} - w_3 r_{comp} + w_4 r_{calib} \quad (26)$$

where:

- r_{acc} : +1 if the response is correct, −1 if wrong, −0.25 if the engine abstained. Correctness is lexical: the normalized reference appears as a whole phrase in a response with no negation word, or word-level F1 between response and reference reaches 0.5. “Paris is not the capital of France” is wrong for reference “Paris”.
- r_{lat} : latency divided by a declared deployment budget (2 s by default)
- r_{comp} : measured compute in excess of a query-only answer: the parameter-token products of every backbone, critic and SSM pass, divided by active parameters times query plus response tokens, minus one

- $r_{calib} = -(c - y_{true})^2$: calibration of the reported confidence c (critic score when verified, token likelihood otherwise, 0 after an abstention); this rewards verifying when verification improves calibration

Weights: $w_1 = 1.0, w_2 = 0.3, w_3 = 0.2, w_4 = 0.5$. Quality-cost curves under several weightings, not one weighted sum, are the intended output.

Supervised warm start. Before PPO, an optional route search evaluates every allowed gate combination for a query on frozen memory and trains the gate logits toward the best-rewarded action. This supplies paired action outcomes before the stochastic policy explores.

PPO Update: The policy is updated by minimizing the clipped surrogate loss:

$$\mathcal{L}^{PPO}(\theta) = -\mathbb{E} \left[\min \left(r(\theta) \hat{A}, \text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A} \right) \right] \quad (27)$$

where $r(\theta) = \frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)}$ is the probability ratio over the joint gate and expert-bias action, $\hat{A} = R - V_{\omega}(\mathbf{e}_q, \mathbf{s})$ is the advantage against a learned value network (normalized per batch), and $\epsilon = 0.2$. Each update runs 4 epochs over a batch of 256 on-policy episodes in minibatches of 64, with learning rate 10^{-5} , gradient clipping at 1.0 and a clipped value loss. Validation reports mean reward, accuracy over all questions, coverage, error rate among answered questions, p95 latency and mean compute.

4.4 Stage 4: Critic Training

The critic is trained with supervision on labeled (query, response, evidence) triples, using AdamW with learning rate 10^{-4} and the loss above, with the base model frozen. The data is split into training, calibration and validation parts; the temperature is fitted on the calibration part and Brier score and ECE are reported on the validation part. We propose curated hallucination datasets (HaluEval [33], FaithDial [34]) as training data, with:

- Positive examples: Factually correct responses with supporting evidence
- Negative examples: Plausible but incorrect outputs with contradictory evidence, including near misses (negated or altered versions of correct answers) and outputs produced by the actual generator

4.5 Stage 5: Generator Adaptation

Stages 2–4 freeze the generator, so better retrieval need not produce better answers. Stage 5 fine-tunes the base model on instruction data in the runtime prompt format: conversation roles, an evidence block whose lines carry source identifiers and provenance labels, and a response. The loss covers only response tokens. Records may include distractor or contradictory evidence and unanswerable questions whose response is an abstention. The saved checkpoint marks the chat format so the engine builds prompts the way the model was trained.

5 Proposed Evaluation Protocol

5.1 Baselines

We propose comparing MANTIS against the following baselines to validate its effectiveness:

- **Llama-3-8B** [3]: Dense transformer baseline
- **Mixtral-8x7B** [17]: Sparse MoE without memory augmentation
- **Mamba-2.8B** [13]: Pure SSM architecture
- **Llama-3-8B + RAG**: Dense model with FAISS retrieval

The comparison that matters most is an ablation ladder over the same backbone: no memory or controller; ordinary retrieval; a recent-text buffer; episodic SSM retrieval; a fixed policy plus the verifier; the learned router; bounded expert bias; learned consolidation. Each rung answers whether the next component earns its cost. An oracle-evidence condition and an oracle route selector on a diagnostic subset separate retrieval failures from generator failures.

5.2 Proposed Datasets

- **LongBench** [35]: Long-context QA (16K-128K tokens)
- **TruthfulQA** [36]: Hallucination-prone questions
- **HaluEval** [33]: Synthetic hallucination detection
- **NarrativeQA** [37]: Book-length comprehension

5.3 Proposed Metrics

1. **Accuracy:** Exact match / F1 score on QA tasks
2. **Error and Abstention:** Error rate over all questions, coverage (fraction answered), error rate among answered questions, and confident-error rate (wrong with confidence ≥ 0.8)
3. **Latency:** Wall-clock inference time (A100 GPU)
4. **FLOPs:** Floating-point operations per token
5. **Calibration Error:** Expected Calibration Error (ECE) [38], Brier score and the area under the risk–coverage curve
6. **Memory:** Recall of facts stored in earlier sessions, correctness after updates, robustness to distractors and abstention on unanswerable questions, in the memory condition and in a recent-text-buffer baseline

5.4 Implemented Harness

The reference implementation evaluates MMLU [30], TruthfulQA [36], HumanEval [31] and GSM8K [32], loaded from the HuggingFace Hub, plus a synthetic multi-session memory benchmark, for either the base model alone or the full engine rebuilt from a Stage 3 policy. Independent-example benchmarks run on frozen memory so results do not depend on evaluation order. The engine’s confidence is the critic score when the answer was verified, the geometric-mean token probability otherwise, and 0 after an abstention; abstentions are recorded. MMLU prompts the question and lettered choices, decodes at most 10 tokens and reads the first standalone letter. TruthfulQA counts a response as truthful when its word F1 is higher against a true reference than against every false one; this is a lexical proxy, not the official judge, and its scores are not comparable with published ones. HumanEval runs generated code in a Docker container without network when Docker is available, else in a resource-limited subprocess that is not a security boundary. Reports keep per-example records (route, evidence identifiers, timings, compute) and artifact versions, and command-line flags switch the route policy, expert bias, serving precision and memory mode for ablations.

6 Expected Properties

6.1 Computational Efficiency

The hierarchical memory design is expected to enable graceful degradation: predictable queries should bypass retrieval and verification (Gate 1), while harder ones activate the memory hierarchy. The bypass path skips optional components only; every path runs the backbone at full depth with the same expert count, so the saving is bounded by retrieval, verification and memory-write costs. For query length Q , evidence length R and output length T , the bypass path costs one prefill of Q plus T decode steps, and the full path one prefill of Q (reused when no evidence is prepended), one prefill of $Q + R$ otherwise, T decode steps, an optional critic pass over up to 2048 tokens, and an optional second round. End-to-end measurements, not parameter counts, decide whether routing pays.

6.2 Scaling Analysis

Attention cost is fixed by the preset’s window; history beyond it reaches the generator only through retrieval. Episodic retrieval is linear in the number of entries (at most 100 keys), and semantic retrieval costs $O(\sqrt{M})$ under the assumptions stated above. Whether this recovers the information a longer window would provide is an empirical question about retrieval recall and evidence budgets, which the memory benchmark and the long-context tests (RULER, LongMemEval) are meant to answer.

7 Discussion

7.1 Architectural Design Decisions

The decision to separate memory into three tiers is motivated by the distinct access patterns required for effective language modeling. Working memory handles immediate syntax and local coherence; episodic memory indexes recent interactions by independent keys; and semantic memory provides access to long-term facts with provenance. The meta-controller serves as a learned arbitrator, aiming to balance the trade-off between retrieval cost and information gain.

7.2 Hallucination Mitigation Strategy

The critic model is designed to improve calibration by conditioning verification on retrieved evidence. By scoring generated outputs against the evidence the generator saw, the system aims to filter out ungrounded generation, and the trust levels keep the generator’s own unverified claims from becoming evidence for its later claims. The expected failure modes involve multi-hop reasoning where the necessary evidence is distributed across memory tiers or entries; one bounded recovery round is the only iteration the engine performs.

7.3 Limitations

- **No Trained Weights:** Every component ships untrained. No benchmark, throughput or calibration result exists; the design claims above remain hypotheses until training and evaluation test them.
- **Measurement Plan:** A reliability review of the implementation set three gates before any large training commitment: a memory gate (beat ordinary retrieval on the memory benchmark at the same generator, context and storage budget, with a paired confidence interval excluding zero), an efficiency gate (lower measured cost at comparable quality without violating a p95 latency target) and a reliability gate (lower error rate among answered questions at matched coverage on held-out sources). The ablation ladder of Section 5 supplies the controls.
- **Training Complexity:** RL optimization of the meta-controller presents a stability challenge and requires careful hyperparameter tuning. Its accuracy reward is lexical and misjudges paraphrases; verbose correct answers can score below the F1 threshold.
- **Memory Maintenance:** Each L3 rebuild retrains the IVF-PQ codebooks over all live entries, a cost that grows faster than linearly in M ; vectors and text live in RAM (8.2 GB of vectors for 1M entries at $d_{model} = 2048$). Disk-backed storage remains future work.
- **Retention Policy:** Consolidation promotes entries by retrieval hits and preserves every evicted record, but it does not learn which facts a later query will need, nor does it extract atomic facts or resolve conflicting updates.
- **Tokenizer Specialization:** The 512-token vocabulary targets the simulation protocol, so natural-language workloads pay for longer sequences: attention over the same text costs more, and a fixed window covers less of it. A subword vocabulary for assistant work has not been evaluated.
- **Cold Start:** The system requires a warmup period to populate the episodic memory before it can leverage historical context.

8 Conclusion

We have presented MANTIS, a proposed metacognitive adaptive network designed to address fundamental trade-offs in long-context LLM design. It combines sparse MoE computation with tiered memory under RL-based meta-control, in the hope of improving both efficiency and factual accuracy. This paper details the architecture, its five-stage training pipeline, its memory lifecycle and its evaluation harness. Stage 1 supports controlled experiments now; the integrated reliability and efficiency claims await the ablations and gates listed above. If the design holds, learned routing policies could allocate retrieval and verification according to query needs.

Future work, in order of what the gates need first: (1) a general-language tokenizer evaluated for bytes per token and end-to-end cost, (2) RULER and LongMemEval runs alongside the synthetic memory benchmark, (3) retention learned from the future usefulness of stored evidence, with atomic facts and supersession, (4) a cost-constrained routing objective (minimize expected cost subject to an answered-error bound, a coverage floor and a p95 latency target), (5) disk-backed vector and text storage, and (6) profiled expert kernels and sharding before any large pretraining run.

Broader Impact

The proposed MANTIS architecture aims to improve reliability in high-stakes applications (medical QA, legal reasoning). However, the critic model might introduce brittleness if trained on biased datasets, potentially amplifying systemic biases. Any future deployment should include human-in-the-loop verification for critical decisions.

References

- [1] T. Brown et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020.
- [2] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” *arXiv:2302.13971*, 2023.
- [3] A. Grattafiori et al., “The Llama 3 Herd of Models,” *arXiv:2407.21783*, 2024.
- [4] J. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” *EMNLP*, 2023.
- [5] Anthropic, “Claude 3 Model Family,” Technical Report, 2024.
- [6] A. Vaswani et al., “Attention is All You Need,” *NeurIPS*, 2017.
- [7] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention,” *NeurIPS*, 2022.
- [8] N. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *TACL*, 2023.
- [9] Y. Zhang et al., “Siren’s Song in the AI Ocean,” *EMNLP*, 2023.
- [10] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” *ACM Comp. Surveys*, 2023.
- [11] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP,” *NeurIPS*, 2020.
- [12] A. Gu et al., “Efficiently Modeling Long Sequences with Structured State Spaces,” *ICLR*, 2022.
- [13] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [14] R. Child et al., “Generating Long Sequences with Sparse Transformers,” *arXiv:1904.10509*, 2019.
- [15] I. Beltagy et al., “Longformer: The Long-Document Transformer,” *arXiv:2004.05150*, 2020.
- [16] W. Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models,” *JMLR*, 2022.
- [17] A. Q. Jiang et al., “Mixtral of Experts,” *arXiv:2401.04088*, 2024.
- [18] O. Lieber et al., “Jamba: A Hybrid Transformer-Mamba Language Model,” *arXiv:2403.19887*, 2024.
- [19] A. Graves et al., “Neural Turing Machines,” *arXiv:1410.5401*, 2014.
- [20] J. Weston et al., “Memory Networks,” *ICLR*, 2015.
- [21] G. Izacard et al., “Atlas: Few-shot Learning with Retrieval Augmented Language Models,” *arXiv:2208.03299*, 2022.
- [22] P. Manakul et al., “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection,” *EMNLP*, 2023.
- [23] L. Gao et al., “RARR: Researching and Revising What Language Models Say,” *ACL*, 2023.
- [24] L. Kuhn et al., “Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation,” *ICLR*, 2023.
- [25] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” *arXiv:1606.08415*, 2016.
- [26] J. Johnson et al., “Billion-scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, 2019.
- [27] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *ICLR*, 2019.
- [28] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [29] J. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *arXiv:2104.09864*, 2021.
- [30] D. Hendrycks et al., “Measuring Massive Multitask Language Understanding,” *ICLR*, 2021.

- [31] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv:2107.03374*, 2021.
- [32] K. Cobbe et al., “Training Verifiers to Solve Math Word Problems,” *arXiv:2110.14168*, 2021.
- [33] Y. Li et al., “HaluEval: A Large-Scale Hallucination Evaluation Benchmark,” *EMNLP*, 2023.
- [34] N. Dziri et al., “FaithDial: A Faithful Benchmark for Information-Seeking Dialogue,” *TACL*, 2022.
- [35] Y. Bai et al., “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding,” *arXiv:2308.14508*, 2023.
- [36] S. Lin et al., “TruthfulQA: Measuring How Models Mimic Human Falsehoods,” *ACL*, 2022.
- [37] T. Kociský et al., “The NarrativeQA Reading Comprehension Challenge,” *TACL*, 2018.
- [38] C. Guo et al., “On Calibration of Modern Neural Networks,” *ICML*, 2017.